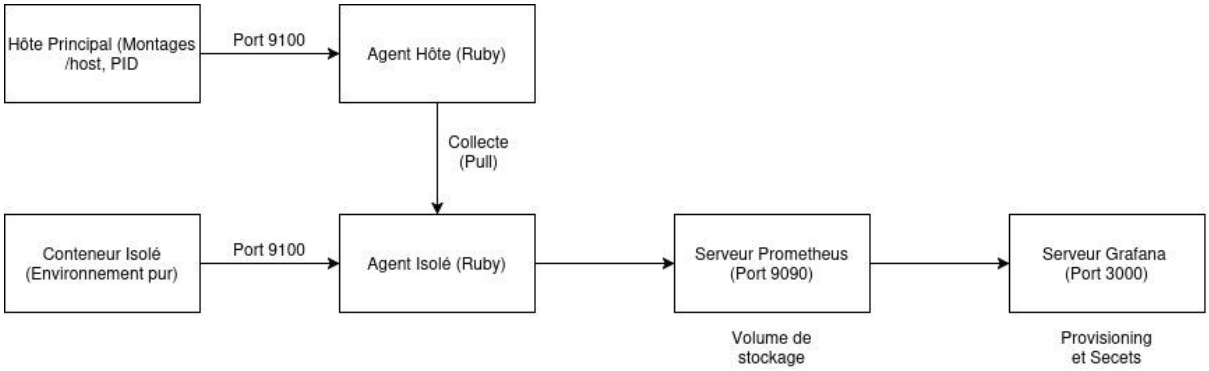
Rapport : Audit Système et Supervision Centralisée

Dépôt du projet : https://github.com/FissonRonan/preSAEsuite

1. Schéma et Analyse de l'Architecture Réalisée

L'infrastructure mise en place repose sur un modèle conteneurisé rigoureux orchestré par Docker Compose. Elle sépare de manière étanche les composants de collecte des métriques, le moteur de stockage centralisé et la couche de visualisation graphique.



- **Le Modèle de Collecte (Pull) :** Prometheus ne dépend pas d'une remontée active des agents, mais interroge périodiquement (*scraping*) deux instances d'agents HTTP en Ruby écoutant sur le port TCP 9100.
- **L'Agent Hôte (agent-hote) :** Conçu pour s'affranchir de l'isolation normale de Docker afin de superviser la machine physique, il est exécuté avec les options avancées pid: "host" et network_mode: "host". Il intègre en outre un montage du système de fichiers racine en lecture seule (/:/host:ro) et exploite le paramètre de chemin -r /host/ pour auditer l'ensemble des métriques de la machine hôte.
- **L'Agent Isolé (agent-isole) :** Déployé sous forme de conteneur standard Debian sans privilèges particuliers, il sert de témoin pour auditer son propre environnement confiné.
- **Le Stockage et la Restitution :** Prometheus (v2.47.0) centralise les séries temporelles dans un volume persistant (prometheus_data). Grafana (v10.1.5) s'y connecte pour alimenter un tableau de bord unifié, avec un chargement automatique (*provisioning*) et une sécurisation stricte des mots de passe administrateur via des **Docker Secrets** (grafana_password.txt ignoré par le gestionnaire de versions).

2. Choix Techniques et Adaptation pour la Supervision

Pour transformer le script d'audit du projet précédent en un véritable exporteur de métriques compatible Prometheus, j'ai dû concevoir et adapter les mécanismes suivants :

- **Mise en place d'un serveur HTTP léger (TCPServer) :** Contrairement au projet précédent où le script s'exécutait ponctuellement avant de s'arrêter, Prometheus fonctionne sur un modèle *pull* et attend un service permanent. J'ai donc intégré un serveur HTTP minimaliste à l'aide de la classe standard TCPServer (bibliothèque standard socket de Ruby). Ce choix permet de répondre directement aux requêtes GET /metrics sans alourdir l'agent avec des dépendances superflues (sachant que WEBrick n'est plus inclus nativement depuis Ruby 3.0).
- **Format d'exposition texte Prometheus :** Les données collectées sont formatées selon les conventions standard, associant un préfixe commun (audit_), des suffixes d'unités explicites (_bytes, _seconds) et des labels discriminants (comme le type de RAM ou le nom du service) plutôt que des chaînes de caractères textuelles non exploitables en séries temporelles.
- **Traitement spécifique du CPU (/proc/stat) :** Contrairement aux métriques instantanées comme la RAM ou le disque, l'utilisation du processeur ne se mesure pas à un instant T. Le fichier /proc/stat fournit des temps cumulés en ticks depuis le démarrage de la machine. L'agent expose donc ces ticks bruts sous forme de compteur (audit_cpu_ticks_total), déléguant au moteur Prometheus le calcul dynamique du taux d'utilisation à l'aide de la fonction rate().
- **Optimisation des performances :** Pour respecter la contrainte de délai de collecte (le scrape_timeout fixé par défaut à 10 secondes dans Prometheus), l'ensemble de la collecte et du traitement exécuté par le script Ruby est optimisé pour s'exécuter en moins de 10 millisecondes, évitant ainsi tout décalage ou faux positif d'indisponibilité (up 0).
- **Automatisation et Sécurité (Bonus) :** L'intégration d'un mécanisme de *provisioning* Grafana permet de charger automatiquement les sources de données et les tableaux de bord JSON au premier lancement via Docker Compose. De plus, les identifiants administrateur ne sont pas inscrits en clair dans les fichiers de configuration, mais transitent par des **Docker Secrets**.

3. Déploiement Docker & Stratégie de Sécurité

Superviser une machine hôte depuis l'intérieur d'un conteneur demande un minimum de rigueur pour garantir la sécurité globale :

- **Briser l'isolation (Conteneur Hôte) :** J'utilise les options pid: "host" et network_mode: "host" pour que l'agent dispose d'une vision réelle des processus et des cartes réseau de la machine physique.
- **Le principe du moindre privilège :** Le disque de l'hôte est monté en strict **lecture seule** (/:/host:ro). Le conteneur peut consulter l'état du système, mais il ne pourra jamais modifier ou corrompre la machine hôte.
- **Gestion des secrets :** Aucun mot de passe n'est écrit en clair dans le code source ou les configurations ; tout transite proprement via les mécanismes natifs de **Docker Secrets**.

4. Les Métriques Exposées

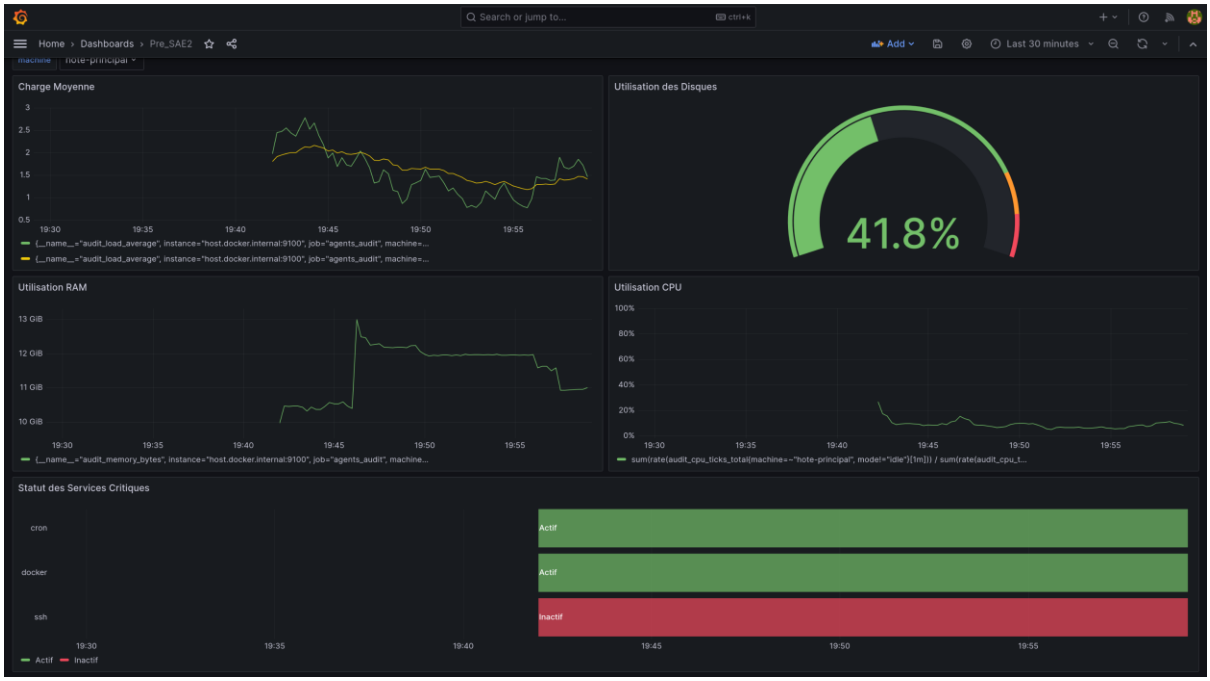
Les informations collectées par le script Ruby sont converties au format d'exposition standard de Prometheus :

- audit_load_average (Type *Gauge*) : La charge moyenne du système sur les fenêtres de 1 et 5 minutes.
- audit_cpu_ticks_total (Type *Counter*) : Le cumul des ticks CPU par mode (user, system, idle), transformé dans Grafana en un pourcentage d'utilisation globale (0-100%).
- audit_memory_bytes (Type *Gauge*) : La mémoire RAM totale et utilisée en octets.
- audit_disk_bytes (Type *Gauge*) : L'espace disque total et utilisé en octets.
- audit_service_active (Type *Gauge*) : L'indicateur d'état binaire des services surveillés (1 si actif, 0 si arrêté ou en erreur).

5. Le Dashboard Grafana

Grâce au mécanisme de *provisioning*, le déploiement de l'infrastructure génère instantanément un tableau de bord complet et unifié comprenant :

- Une vue temporelle de la **Charge Moyenne**.
- Une **jauge dynamique** pour l'**Utilisation des Disques** (avec seuils d'alerte en couleur).
- L'évolution en temps réel de l'**Utilisation de la RAM**.
- Le graphique d'**Utilisation CPU** en pourcentage.
- Une frise chronologique (**State Timeline**) affichant en clair l'activité des services critiques (ssh, cron, docker).



6. Limites et Perspectives

6. Limites et Perspectives

- **Passage à l'échelle :** Notre solution fonctionne très bien pour surveiller une ou deux machines de test. Par contre, si on devait gérer un grand parc de centaines de serveurs, centraliser toutes les requêtes sur un seul serveur Prometheus pourrait saturer le réseau. Il faudrait alors mettre en place une organisation en plusieurs niveaux (fédération).
- **Sécurité des données :** Pour l'instant, les métriques circulent en clair sur le port 9100 sans mot de passe ni chiffrement. N'importe qui sur le réseau local peut les lire. En production, il faudrait ajouter un système de chiffrement (HTTPS) et une authentification.
- **Automatisation :** Tout est actuellement assemblé proprement avec Docker Compose. Pour industrialiser l'installation sur des serveurs physiques distants, l'étape suivante serait d'utiliser un outil d'automatisation comme Ansible.